

MCT 332/MCT334 Design of MCT 2

Spring 2026

ROS2 Navigation



Lab Objectives:

I. Develop Basic Obstacle Avoidance

- Create a Python ROS2 node using LaserScan data.
- Implement simple reactive navigation behavior.

II. Develop Waypoints Control

- Create a Python node controlling the mobile robot using odometry and imu data.
- Implement the waypoints for the mobile robot.

Steps:

➤ Obstacle Avoidance Python Node

You can create a python node for your robot to detect the obstacle and avoid it.

- I. Create a new folder named scripts in *mobile_robot_sim* package. Then create a python file.

```
cd ~/design_ws/src/mobile_robot_sim && mkdir scripts
```

```
cd scripts && touch obstacle_avoidance.py
```

- II. Add the following code in the python file

```
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan
from geometry_msgs.msg import TwistStamped
import numpy as np

class ObstacleAvoidance(Node):

    def __init__(self):
        super().__init__('obstacle_avoidance')

        self.subscription = self.create_subscription(
            LaserScan,
            '/scan',
            self.scan_callback,
            10)

        self.publisher = self.create_publisher(
            TwistStamped,
            '/diff_drive_controller/cmd_vel',
            10)

        self.safe_distance = 0.5
        self.get_logger().info("Obstacle Avoidance Node Started")

    def scan_callback(self, msg):

        ranges = np.array(msg.ranges)
        ranges = np.nan_to_num(ranges, nan=10.0, posinf=10.0)

        if len(ranges) == 0:
            return

        front_ranges = np.concatenate((ranges[:30], ranges[-30:]))

        min_distance = np.min(front_ranges)

        cmd = TwistStamped()
        cmd.header.stamp = self.get_clock().now().to_msg()
        cmd.header.frame_id = "RobotBody"

        if min_distance < self.safe_distance:
            cmd.twist.linear.x = 0.0
            cmd.twist.angular.z = 0.5
            self.get_logger().info("Obstacle detected -> Turning")
        else:
            cmd.twist.linear.x = 0.2
            cmd.twist.angular.z = 0.0
            self.get_logger().info("Path clear -> Moving forward")

        self.publisher.publish(cmd)

def main(args=None):
    rclpy.init(args=args)
    node = ObstacleAvoidance()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

III. Make the python file executable

chmod +x obstacle_avoidance.py

IV. Add **scripts** in the share section and make obstacle avoidance node ROS executable node in CMakeList.txt

```
install(
  DIRECTORY urdf meshes launch rviz config world
  scripts
  DESTINATION share/${PROJECT_NAME}
)
install(
  PROGRAMS
  scripts/obstacle_avoidance.py
  DESTINATION lib/${PROJECT_NAME}
)
```

V. Add the following dependencies in package.xml

```
<depend>roscpp</depend>
<depend>geometry_msgs</depend>
<depend>sensor_msgs</depend>
```

VI. Build workspace, then launch the file.

```
cd ~/design_ws

colcon build --symlink-install --packages-select
mobile_robot_sim

source install/setup.bash

ros2 launch mobile_robot_sim robot_gazebo.launch.py
```

VII. Open another terminal and run the python node.

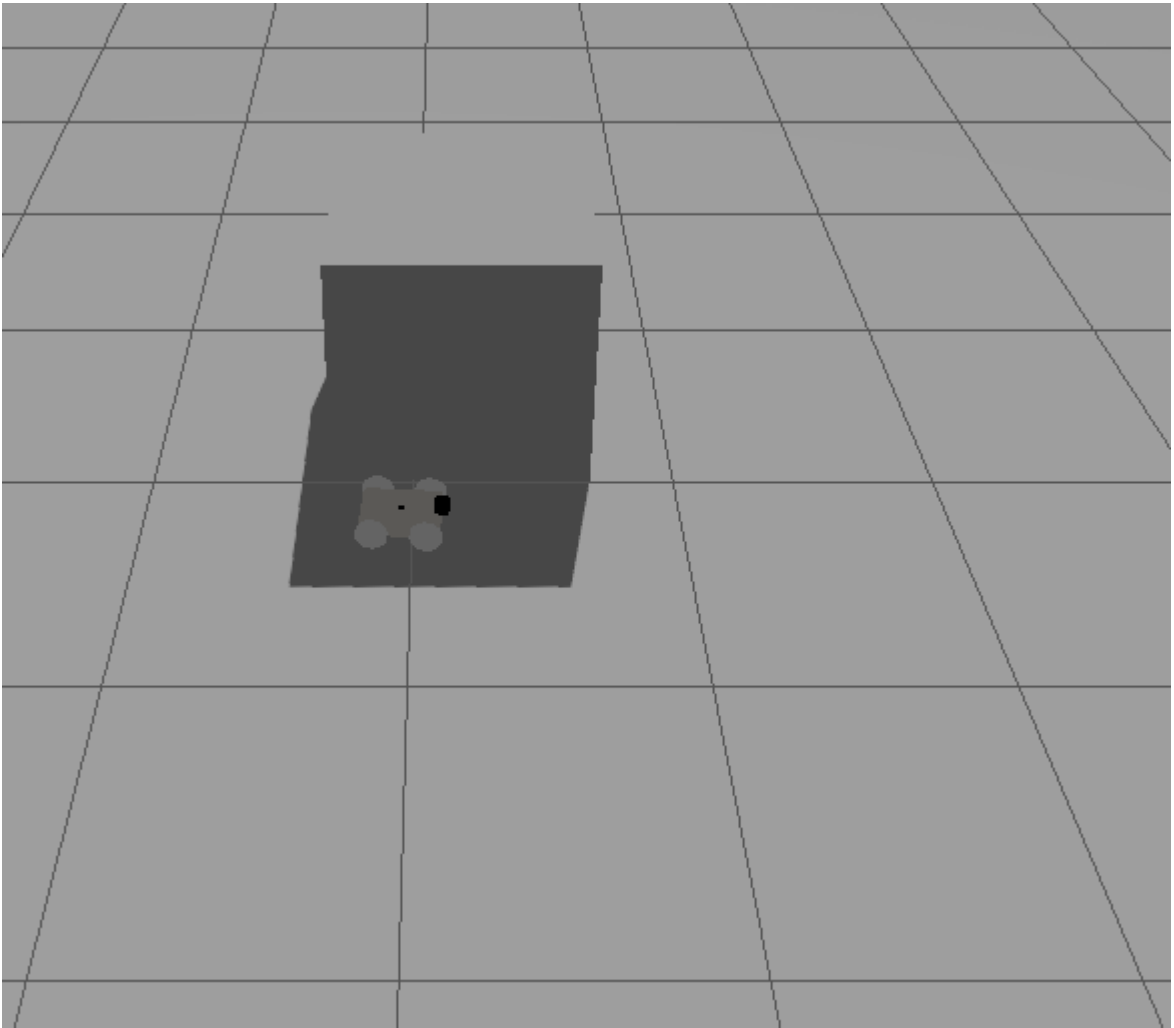
```
cd ~/design_ws

source install/setup.bash

ros2 run mobile_robot_sim obstacle_avoidance.py
```

```
jina2997@DESKTOP-E9CU3F0:~/design_ws$ ros2 run robot_gazebo obstacle_avoidance.py
[INFO] [1771524192.418808176] [obstacle_avoidance]: Obstacle Avoidance Node Started
[INFO] [1771524192.421102457] [obstacle_avoidance]: Path clear -> Moving forward
[INFO] [1771524192.522707515] [obstacle_avoidance]: Path clear -> Moving forward
[INFO] [1771524192.600110973] [obstacle_avoidance]: Path clear -> Moving forward
[INFO] [1771524192.713840230] [obstacle_avoidance]: Path clear -> Moving forward
[INFO] [1771524192.797756935] [obstacle_avoidance]: Path clear -> Moving forward
[INFO] [1771524192.892159052] [obstacle_avoidance]: Path clear -> Moving forward
[INFO] [1771524192.955875824] [obstacle_avoidance]: Path clear -> Moving forward
[INFO] [1771524193.026812242] [obstacle_avoidance]: Path clear -> Moving forward
[INFO] [1771524193.066896820] [obstacle_avoidance]: Path clear -> Moving forward
```

```
[INFO] [1771524221.179160949] [obstacle_avoidance]: Obstacle detected -> Turning
[INFO] [1771524221.200707364] [obstacle_avoidance]: Obstacle detected -> Turning
[INFO] [1771524221.225957782] [obstacle_avoidance]: Obstacle detected -> Turning
[INFO] [1771524221.277370018] [obstacle_avoidance]: Obstacle detected -> Turning
[INFO] [1771524221.324662051] [obstacle_avoidance]: Obstacle detected -> Turning
[INFO] [1771524221.369373882] [obstacle_avoidance]: Obstacle detected -> Turning
[INFO] [1771524221.421082219] [obstacle_avoidance]: Obstacle detected -> Turning
[INFO] [1771524221.446748437] [obstacle_avoidance]: Obstacle detected -> Turning
[INFO] [1771524221.493503870] [obstacle_avoidance]: Obstacle detected -> Turning
[INFO] [1771524221.537468801] [obstacle_avoidance]: Obstacle detected -> Turning
[INFO] [1771524221.562351018] [obstacle_avoidance]: Obstacle detected -> Turning
[INFO] [1771524221.621107459] [obstacle_avoidance]: Obstacle detected -> Turning
```



➤ *Python Controlled Waypoints*

I. Create a python file in scripts folder

```
cd ~/design_ws/src/mobile_robot_sim/scripts
```

```
touch waypoints_control.py
```

```
chmod +x waypoints_control.py
```

II. Define the waypoints for the robot

```
#Defined Waypoints
waypoints = [
    (0.0, 0.0, 0.0),
    (1.0, 0.0, 0.0),
    (1.0, 1.0, math.pi / 2),
    (0.0, 1.0, math.pi)
]
```

III. Define the reading method of the current position from the odometry and orientation from the IMU

```

current_time = node.get_clock().now()

if last_time is None:
    last_time = current_time
    return
#get the time difference
dt = (current_time - last_time).nanoseconds / 1e9
last_time = current_time

v = linear_vel

#calculate the distance moved in x and y
delta_x = v * math.cos(yaw) * dt
delta_y = v * math.sin(yaw) * dt

#update the current position of the robot
x += delta_x
y += delta_y
yaw = normalize_angle(yaw)

```

IV. Define the control method as state machine

```

#get the difference in current and target pose
dx = target_x - x
dy = target_y - y

distance = math.sqrt(dx**2 + dy**2)

#the angle to be rotated to the target pose
angle_to_target = math.atan2(dy, dx)
angle_error = normalize_angle(angle_to_target - yaw)

#calculate the difference in target angle and current angle
final_yaw_error = normalize_angle(target_yaw - yaw)

```

```

#set the configurations of the cmd_vel
cmd = TwistStamped()
cmd.header.stamp = node.get_clock().now().to_msg()
cmd.header.frame_id = "RobotBody"

#State Machine
#First rotate to the target pose direction
if state == "rotate_to_target":
    node.get_logger().info("rotate_to_target state")

    if abs(angle_error) > 0.005:
        cmd.twist.angular.z = -1.5 * angle_error
    else:
        state = "move_forward"

#Second move forward to the pose
elif state == "move_forward":
    node.get_logger().info("move_forward state")

    if distance > 0.05:
        cmd.twist.linear.x = 0.5 * distance
    else:
        state = "rotate_to_final"

#Third rotate to the final target angle
elif state == "rotate_to_final":
    node.get_logger().info("rotate_to_final state")

    if abs(final_yaw_error) > 0.005:
        cmd.twist.angular.z = -1.5 * final_yaw_error
    else:
        current_waypoint = (current_waypoint + 1) %
len(waypoints)
        state = "rotate_to_target"
#send the velocity command
cmd_pub.publish(cmd)

```

Final Code

```

#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from geometry_msgs.msg import TwistStamped
from nav_msgs.msg import Odometry
from rosgraph_msgs.msg import Clock
from sensor_msgs.msg import Imu
import math
import tf_transformations

# Add all the global variables
yaw = 0.0
roll = 0.0
pitch = 0.0

x = 0.0
y = 0.0

linear_vel = 0.0
angular_vel = 0.0

last_time = None

#Defined Waypoints
waypoints = [
    (0.0, 0.0, 0.0),
    (1.0, 0.0, 0.0),
    (1.0, 1.0, math.pi / 2),
    (0.0, 1.0, math.pi)
]

current_waypoint = 0
state = "rotate_to_target"

node = None
cmd_pub = None

# normalize the angle between -pi to pi
def normalize_angle(angle):
    return math.atan2(math.sin(angle), math.cos(angle))

#IMU topic callback function
def imu_callback(msg):
    global yaw, roll, pitch

    q = msg.orientation
    quaternion = (q.x, q.y, q.z, q.w)
    roll, pitch, yaw_val = tf_transformations.euler_from_quaternion(quaternion)

    roll = roll
    pitch = pitch
    yaw = yaw_val

#odom topic callback function
def odom_callback(msg):
    global linear_vel, angular_vel

    linear_vel = msg.twist.twist.linear.x
    angular_vel = msg.twist.twist.angular.z

```

```

#clock topic callback function
def clock_callback(msg):
    global x, y, yaw

    sim_time = msg.clock
    if sim_time.sec == 0:
        x = 0.0
        y = 0.0
        yaw = 0.0

def control_loop():
    global x, y, yaw
    global linear_vel, angular_vel
    global last_time
    global current_waypoint, state

    node.get_logger().info("Control loop")

    current_time = node.get_clock().now()

    if last_time is None:
        last_time = current_time
        return
    #get the time difference
    dt = (current_time - last_time).nanoseconds / 1e9
    last_time = current_time

    v = linear_vel
    w = angular_vel

    #calculate the distance moved in x and y
    delta_x = v * math.cos(yaw) * dt
    delta_y = v * math.sin(yaw) * dt
    delta_yaw = w * dt

    #update the current position of the robot
    x += delta_x
    y += delta_y
    yaw += delta_yaw
    yaw = normalize_angle(yaw)

    #print the position, orientation and velocity of the robot
    node.get_logger().info(
        f"Position -> x: {x:.3f}, y: {y:.3f} | "
        f"yaw: {math.degrees(yaw):.2f}° | "
        f"Vel -> linear: {linear_vel:.3f}, "
        f"angular: {angular_vel:.3f}"
    )

    #get the target waypoint
    target_x, target_y, target_yaw = waypoints[current_waypoint]

    #get the difference in current and target pose
    dx = target_x - x
    dy = target_y - y

    distance = math.sqrt(dx**2 + dy**2)

    #the angle to be rotated to the target pose
    angle_to_target = math.atan2(dy, dx)
    angle_error = normalize_angle(angle_to_target - yaw)

    #calculate the difference in target angle and current angle
    final_yaw_error = normalize_angle(target_yaw - yaw)

    node.get_logger().info(
        f"Target Pose -> x: {target_x:.3f}, y: {target_y:.3f}, yaw: {target_yaw:.3f}"
    )
    node.get_logger().info(f'Final Yaw Error -> {final_yaw_error:.3f}')

    #set the configurations of the cmd_vel
    cmd = TwistStamped()
    cmd.header.stamp = node.get_clock().now().to_msg()
    cmd.header.frame_id = "RobotBody"

```



```

#State Machine
#First rotate to the target pose direction
if state == "rotate_to_target":
    node.get_logger().info("rotate_to_target state")

    if abs(angle_error) > 0.005:
        cmd.twist.angular.z = -1.5 * angle_error
    else:
        state = "move_forward"

#Second move forward to the pose
elif state == "move_forward":
    node.get_logger().info("move_forward state")

    if distance > 0.05:
        cmd.twist.linear.x = 0.5 * distance
    else:
        state = "rotate_to_final"

#Third rotate to the final target angle
elif state == "rotate_to_final":
    node.get_logger().info("rotate_to_final state")

    if abs(final_yaw_error) > 0.005:
        cmd.twist.angular.z = -1.5 * final_yaw_error
    else:
        current_waypoint = (current_waypoint + 1) % len(waypoints)
        state = "rotate_to_target"
#send the velocity command
cmd_pub.publish(cmd)

def main():
    global node, cmd_pub, last_time

    rclpy.init()

    node = Node('square_follower')

    cmd_pub = node.create_publisher(
        TwistStamped,
        '/diff_drive_controller/cmd_vel',
        10
    )

    node.create_subscription(Imu, '/imu', imu_callback, 10)
    node.create_subscription(Odometry, '/diff_drive_controller/odom', odom_callback, 10)
    node.create_subscription(Clock, '/clock', clock_callback, 10)

    node.create_timer(0.05, control_loop)

    last_time = node.get_clock().now()

    node.get_logger().info("Robot State Printer Started")
    node.get_logger().info("Waypoint control Node Started")

    rclpy.spin(node)

    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

V. Add the python file in CMakeLists.txt

```
install(  
  DIRECTORY urdf meshes launch rviz  
  config world scripts  
  DESTINATION share/${PROJECT_NAME}  
)  
install(  
  PROGRAMS  
    scripts/obstacle_avoidance.py  
    scripts/waypoints_final.py  
  DESTINATION lib/${PROJECT_NAME}  
)
```

VI. Build the package, launch your robot and then run the python file in another command

```
cd ~/design_ws
```

```
colcon build --symlink-install --packages-select  
mobile_robot_sim
```

```
source install/setup.bash
```

```
ros2 launch mobile_robot_sim robot_gazebo.launch.py
```

Open another command

```
cd ~/design_ws
```

```
source install/setup.bash
```

```
ros2 run mobile_robot_sim waypoints_final.py
```

